



金融智能体设计案例

当马科维茨遭遇现实：一个量化研究员的蜕变之路

上海财经大学《金融智能体设计》课程教学案例 核心方法：Fan, Li & Yu (2012) · JASA | 智能体框架：LangChain



学习目标

跟着这个故事走完一遍，你将掌握：

#	知识点	核心内容
1	LangChain Tool	用 <code>@tool</code> 装饰器把 Python 函数变成 AI 可调用的工具
2	LangChain Agent	用 <code>create_agent</code> 组装「大脑 + 工具」的完整智能体
3	马科维茨理论	均值-方差优化的原理、代码实现与局限性
4	高维协方差难题	为什么股票多了之后，经典方法会失效？
5	Fan et al. (2012)	用高频数据突破低维瓶颈——前沿方法的故事

 **阅读指南：**本案例采用「边听故事，边跑代码」的形式。请按顺序运行每个代码格，感受主人公的命运起伏，同时理解每一行代码背后的设计逻辑。



登场人物

人物	身份	性格
谭博士	上财金融学博士，正思资本量化研究员，本故事的主角	年轻气盛，相信数学的力量，不撞南墙不回头
林总	正思资本投资总监，谭博士的直属上司	表面温文尔雅，内心如同钢铁，每一句话都算过利息
苏阿姨	正思资本最大个人客户，5000万委托人	做过实业，见过风浪，雷厉风行，不容一点马虎
张教授	上财统计学院教授，谭博士的博士生导师	正当盛年，研究统计出身，说话永远比你想的少一句，却每一句都在点子上

第一章：意气风发

2023年1月，上海·陆家嘴，正思资本22楼

上海的冬天总是阴冷而潮湿，灰色的云压得很低，连东方明珠都只剩下一个模糊的轮廓。

但这些，谭博士完全顾不上看。

他的眼睛死死盯着两块显示器，屏幕的蓝白光打眼镜片上。键盘噼啪声断断续续，最后在一阵急促的敲击后，戛然而止。

```
智能体初始化完成。  
已注册工具: ['download_stock_data', 'markowitz_optimize']
```

搞定了。

谭博士靠回椅背，长出一口气。三个月，他把博士论文里关于马科维茨投资组合的全套理论，硬生生地用 LangChain 封装成了一个 AI 智能顾问系统。给它一句自然语言，它能自己下载数据、优化权重、输出报告——**全程不需要人工干预**。

脑海中忽然浮现出去年毕业答辩后，和张教授在校园咖啡馆的那次谈话。

张教授把咖啡杯推到一边，缓缓说：「你的论文把马科维茨研究得很透彻。但谭博士，理论在象牙塔里是完美的，进了市场，你会遇到它的边界。记住这句话——用样本估计总体，观测越少，风险越大。」

谭博士那时点头如捣蒜，心里想：边界？我把优化算法调了几十遍了，哪有什么边界。

现在回想起来，那个点头实在是太轻浮了。

林总走过来，手里端着咖啡，眼神里什么都没有：「苏阿姨那边，谈好了。五千万，全权委托给你的系统。」

谭博士愣了一秒，随即脑子里的某根弦开始紧绷。

「年底，」林总抿了一口咖啡，「如果业绩不达预期.....你自己想想后果。」

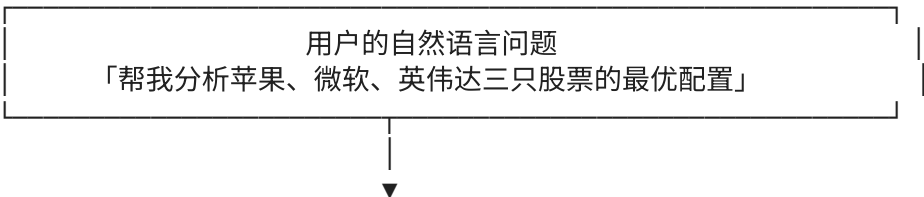
办公室里的空调低鸣着，谭博士感觉手心有点凉。

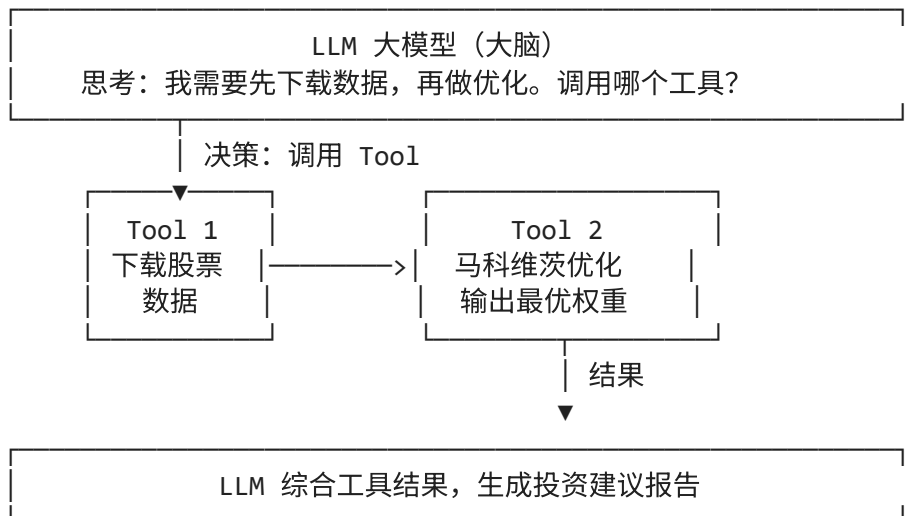
五千万。这不是模拟盘，这是真实的钱，是苏阿姨半辈子的心血。

🎓 **同学们注意！** 接下来，就是谭博士当时搭建的 LangChain 智能系统。让我们跟着他的视角，从零开始理解 Agent 的设计逻辑。

LangChain Agent 核心设计逻辑

谭博士在白板上画了一张架构图，用来向林总解释这套系统：





一句话理解 Agent

Agent = LLM（大脑） + Tools（工具集）

- **LLM** 负责「想」：理解用户意图，决定调用哪个工具、传什么参数
- **Tool** 负责「做」：执行具体的计算、数据获取等操作
- **Agent** 负责「连」：把大脑和工具串联起来，形成完整的推理-行动循环

什么是 Tool?

Tool 就是一个加了 `@tool` 装饰器的普通 Python 函数。关键在三点：

要素	作用	举例
函数签名	告诉 LLM 需要哪些参数及其类型	<code>stock_codes: str, market: str</code>
Docstring	告诉 LLM 工具的用途、参数含义、何时调用	<code>'''下载股票历史行情数据...'''</code>
返回值	工具执行结果，原文发回给 LLM 做下一步判断	JSON 字符串

⚠ **特别注意：** Docstring 写得好不好，直接决定 LLM 能不能正确使用你的工具！这是 Agent 开发中最容易被忽视，却最重要的地方。

```
In [ ]: # — 环境准备 —————
# 如果缺少依赖，先运行：
# pip install langchain langchain-deepseek yfinance scipy python-dotenv

import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from dotenv import load_dotenv
from scipy.optimize import minimize
import yfinance as yf
```

```

from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langchain_deepseek import ChatDeepSeek
from langchain.agents import create_agent

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu S
matplotlib.rcParams['axes.unicode_minus'] = False

print('✅ 所有依赖导入成功')

```

```

In [ ]: # — 初始化大模型 (LLM) —————
# 谭博士选择了 DeepSeek——国内访问稳定、推理能力强

load_dotenv() # 从 .env 文件读取 DEEPSEEK_API_KEY

llm = ChatDeepSeek(
    model='deepseek-chat',
    api_key=os.getenv('DEEPSEEK_API_KEY'),
    temperature=0, # temperature=0: 确定性输出, 金融分析需要可复现
)

print(f'✅ LLM 初始化成功, 模型: {llm.model_name}')

```

第一步：定义工具 (Tool) ——Agent 的「手」

谭博士为系统设计了两个工具，分别负责「数据获取」和「投资组合优化」。

注意观察 `@tool` 装饰器和 **Docstring** 的写法——这是 LLM 理解工具的唯一来源。

```

In [ ]: # — 工具1：股票数据下载 —————
# 全局状态：用于在工具之间共享下载好的数据
_portfolio_data: dict = {}

def _to_yahoo(code: str) -> str:
    '''将6位A股代码转为 Yahoo Finance 格式。'''
    if code.startswith('6'): return f'{code}.SS'
    if code.startswith(('0', '3')): return f'{code}.SZ'
    return code

def _download_raw(codes: list, start_date: str, end_date: str):
    '''yfinance 下载收盘价并计算日收益率。'''
    start = pd.to_datetime(start_date, format='%Y%m%d').strftime('%Y-%m-%d')
    end = pd.to_datetime(end_date, format='%Y%m%d').strftime('%Y-%m-%d')
    raw = yf.download(codes, start=start, end=end, progress=False, auto_adjust=True)
    if raw.empty:
        return json.dumps({'错误': 'yfinance 返回空数据, 请检查代码和日期'}, ensure_ascii=False)
    if isinstance(raw.columns, pd.MultiIndex):
        close = raw['Close']
    else:
        close = raw[['Close']].rename(columns={'Close': codes[0]})
    if isinstance(close, pd.Series):
        close = close.to_frame(name=codes[0])
    close.columns = [str(c) for c in close.columns]

```

```

return close.pct_change().dropna()

#          ↓↓↓ @tool 装饰器：把普通函数变成 AI 可调用的工具 ↓↓↓
@tool
def download_stock_data(
    stock_codes: str,
    market: str = 'A',
    start_date: str = '20240101',
    end_date: str = '20241231',
) -> str:
    '''下载股票历史行情数据并计算日收益率，支持 A 股和美股。

    Args:
        stock_codes: 股票代码，多只用英文逗号分隔。
            A股示例：000001,600519,300750
            美股示例：AAPL,MSFT,GOOGL
        market: 市场选择，A 表示 A 股，US 表示美股，默认 A
        start_date: 起始日期，格式 YYYYMMDD，默认 20240101
        end_date: 结束日期，格式 YYYYMMDD，默认 20241231

    Returns:
        数据下载状态及基本统计摘要（JSON 字符串）
        ...

    codes = [c.strip() for c in stock_codes.split(',')]
    if market.upper() == 'A':
        yahoo_codes = [_to_yahoo(c) for c in codes]
        returns_df = _download_raw(yahoo_codes, start_date, end_date)
        if isinstance(returns_df, pd.DataFrame):
            returns_df = returns_df.rename(columns=dict(zip(yahoo_codes, codes)))
        else:
            returns_df = _download_raw(codes, start_date, end_date)

    if isinstance(returns_df, str): # 错误时返回错误消息字符串
        return returns_df

    _portfolio_data['returns'] = returns_df
    _portfolio_data['stocks'] = codes
    _portfolio_data['market'] = market.upper()

    summary = {
        '状态': '下载成功',
        '市场': 'A股' if market.upper() == 'A' else '美股',
        '股票代码': codes,
        '有效交易日数': len(returns_df),
        '日期范围': f'{str(returns_df.index[0])} 至 {str(returns_df.index[-1])}',
        '各股年化收益率（估算）': {code: f'{returns_df[code].mean() * 252:.2%}' for code in codes},
        '提示': '数据已保存，请调用 markowitz_optimize 进行优化',
    }
    return json.dumps(summary, ensure_ascii=False, indent=2)

print('✅ Tool 1 [download_stock_data] 注册成功')
print('    Docstring 首行: ', download_stock_data.description[:55], '...')

```

In []: # — 工具2：马科维茨投资组合优化 —————

```

@tool
def markowitz_optimize(risk_free_rate: float = 0.02) -> str:
    '''对已下载的股票数据进行马科维茨投资组合优化。

```

同时求解两种最优组合：

- 最大夏普比率组合：风险调整后收益最优，适合积极型投资者
- 最小方差组合：波动率最低，适合保守型投资者

请先调用 `download_stock_data` 下载数据后再使用此工具。

Args:

`risk_free_rate`: 年化无风险利率，默认 0.02 (2%)。美股建议 0.045。

Returns:

两种最优组合的权重及风险收益指标 (JSON 字符串)

...

if 'returns' not in _portfolio_data:

return '错误：请先调用 download_stock_data 下载数据。'

returns_df = _portfolio_data['returns']

stocks = _portfolio_data['stocks']

n = len(stocks)

— 核心数学：均值向量 + 协方差矩阵 —————

mean_ret = returns_df.mean().values # 日均收益率向量

cov_mat = returns_df.cov().values * 252 # 年化协方差矩阵

—————

constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]

bounds = [(0.0, 1.0)] * n

w0 = np.ones(n) / n

def neg_sharpe(w):

ret = float(np.dot(w, mean_ret) * 252)

vol = float(np.sqrt(w @ cov_mat @ w))

return -(ret - risk_free_rate) / vol

def port_vol(w):

return float(np.sqrt(w @ cov_mat @ w))

res_sr = minimize(neg_sharpe, w0, method='SLSQP', bounds=bounds, constraints

res_mv = minimize(port_vol, w0, method='SLSQP', bounds=bounds, constraints

def stats(w):

ret = float(np.dot(w, mean_ret) * 252)

vol = float(np.sqrt(w @ cov_mat @ w))

return ret, vol, (ret - risk_free_rate) / vol

sr_ret, sr_vol, sr_sharpe = stats(res_sr.x)

mv_ret, mv_vol, mv_sharpe = stats(res_mv.x)

result = {

'最大夏普比率组合 (激进型)': {

'权重配置': {stocks[i]: f'{res_sr.x[i]:.2%}' for i in range(n)},

'年化预期收益': f'{sr_ret:.2%}',

'年化波动率': f'{sr_vol:.2%}',

'夏普比率': round(sr_sharpe, 4),

},

'最小方差组合 (保守型)': {

'权重配置': {stocks[i]: f'{res_mv.x[i]:.2%}' for i in range(n)},

'年化预期收益': f'{mv_ret:.2%}',

'年化波动率': f'{mv_vol:.2%}',

'夏普比率': round(mv_sharpe, 4),

```

    },
    '参数说明': f'无风险利率 {risk_free_rate:.1%}, 基于 {len(returns_df)} 个交易周'
}
return json.dumps(result, ensure_ascii=False, indent=2)

print('✅ Tool 2 [markowitz_optimize] 注册成功')

```

🤖 第二步：构建智能体（Agent）——把大脑和工具连起来

工具定义好了，接下来是最关键的一步：**写系统提示词（System Prompt）并创建 Agent。**

System Prompt 就像给这个 AI 员工写的「岗位职责书」——告诉它：

1. 你的角色是什么
2. 遇到问题应该按什么流程处理
3. 有哪些注意事项

谭博士的体会：写好 System Prompt，比写工具代码还难。这是「与 AI 沟通」的艺术，也是 Agent 工程的核心竞争力。

```

In [ ]: # — 创建 Agent —————
SYSTEM_PROMPT = '''你是一位专业的股票投资顾问，擅长运用马科维茨现代投资组合理论进行资产配置。

工作流程（必须严格按顺序执行）：
1. 判断用户要分析的市场（A股 or 美股），调用 download_stock_data 时传入正确的 market：
   - A 股代码示例：000001, 600519, 300750
   - 美股代码示例：AAPL, MSFT, NVDA
2. 调用 markowitz_optimize 进行投资组合优化
   - A 股无风险利率用 0.02（国内国债）
   - 美股无风险利率用 0.045（美国国债）
3. 根据优化结果，从「风险收益特征」和「适合人群」两个维度给出配置建议

注意：本分析仅为教学演示，不构成实际投资建议。'''

# ↓↓↓ create_agent：把 LLM + 工具列表 + 角色设定组装成完整的 Agent ↓↓↓
agent = create_agent(
    model=llm,
    tools=[download_stock_data, markowitz_optimize],
    system_prompt=SYSTEM_PROMPT,
)

print('✅ Agent 创建成功！')
print(f'已注册工具：{[t.name for t in [download_stock_data, markowitz_optimize]}')

```

🚀 第三步：启动！谭博士的系统第一次运行

2023年2月，春节后第一个交易周。

谭博士在会议室里，向苏阿姨做首次演示。他的手指停在回车键上，深呼吸。

「苏阿姨，您只需要用自然语言告诉系统您想怎么配置，它会自动完成所有分析。」

苏阿姨往后靠了靠，说：「那就分析我最熟悉的几只股票——贵州茅台、招商银行、宁德时代，2023年的A股组合，怎么配最好？」

谭博士按下了回车。

 运行下方代码格，看看 Agent 如何一步步推理并给出答案！

Agent 内部的推理循环（ReAct pattern）：

1. LLM 读取问题 → 决定调用 `download_stock_data`
2. 工具执行 → 返回数据摘要给 LLM
3. LLM 决定调用 `markowitz_optimize`
4. 工具执行 → 返回优化结果给 LLM
5. LLM 综合所有信息 → 生成最终投资建议

```
In [ ]: # — 调用 Agent —————
query = '''请帮我分析以下3只A股的投资组合配置：
- 贵州茅台（600519）
- 招商银行（600036）
- 宁德时代（300750）

使用2023年全年数据，进行马科维茨投资组合优化，并给出配置建议。'''

print('👤 用户提问：', query[:50], '...')
print('⌚ Agent 思考中（约20-40秒）...\n')

result = agent.invoke({'messages': [HumanMessage(content=query)]})
print(result['messages'][-1].content)
```

苏阿姨看完报告，点了点头：「不错，有模有样的。」

林总站在一旁，面无表情地说：「希望年底数字也这么好看。」

谭博士笑着送走了两位，转回办公桌，在笔记本上写下：

「2023年2月14日，系统上线。马科维茨，从未让我失望过。」

写完，他顿了顿，想起张教授说过的那句话——「记住它成立的条件」——然后把笔盖上了。

他觉得那句话是废话。

他不知道，命运正在等待最好的时机，准备给他上一堂价值连城的课。

第二章：晴天霹雳

2023年8月，上海，正思资本

那个电话，是在一个周五下午4点17分打来的。

苏阿姨的声音从电话里传来，没有了上次演示时的和蔼：「谭博士，你给我解释一下，这个月的报告怎么是这个数字？」

谭博士盯着屏幕——**组合净值本月回撤 11.3%，超出预警线。**

他感觉血往脸上涌，耳朵嗡嗡的。

「苏阿姨，市场近期波动较大——」

「我不想听这些废话。」苏阿姨打断了他，「你那个 AI 系统，到底是怎么估的风险？」

电话挂断后，谭博士在空荡荡的办公室里坐了很久。

那天晚上，他没有回家。他把近半年的数据、回测结果、协方差矩阵的每一个参数，重新翻了一遍。

凌晨两点，他终于意识到了问题所在。

问题出在哪里？——高维协方差矩阵的「维度诅咒」

事情的起因，要从他 8 月初做的一个决定说起。

苏阿姨想要更分散的配置，于是谭博士把股票池从 **10 只** 扩大到了 **50 只**。

这在理论上完全合理——更多资产，更好的分散化。

但他忽略了一个关键问题：**协方差矩阵的估计质量，随着维度的增加而急剧下降。**

马科维茨优化的核心，是这个 $p \times p$ 的协方差矩阵 $\hat{\Sigma}$ ：

$$\hat{\Sigma} = \frac{1}{T-1} \sum_{t=1}^T (r_t - \bar{r})(r_t - \bar{r})^\top$$

其中 p 是股票数量， T 是观测天数（通常是 252 个交易日/年）。

当 $p = 50$, $T = 252$ 时：

- 参数数量： $p(p+1)/2 = 1275$ 个协方差参数
- 但观测数只有 252 个——**严重欠定！**
- 更糟的是，当 $p > T$ 时，**样本协方差矩阵直接变成奇异矩阵，无法求逆！**

下面，让我们用数据亲眼看看这个「维度诅咒」有多可怕：

In []:

```
# — 演示：高维协方差矩阵的「维度诅咒」——  
# 谭博士当晚重现了这个实验，看到结果后，他手抖了。  
  
np.random.seed(42)
```

```

T = 252 # 固定观测期：一年的交易日

dimensions = [5, 10, 20, 50, 100, 200]
cond_numbers = []
ranks = []

print(f'固定观测期 T = {T} 个交易日 (约1年) \n')
print(f'{"股票数 p":>8} {"p/T":>6} {"条件数":>14} {"矩阵秩":>8} {"是否满秩":>8}')
print('-' * 55)

for p in dimensions:
    R = np.random.randn(T, p) * 0.01
    Sigma_hat = np.cov(R.T)
    cond = np.linalg.cond(Sigma_hat)
    rank = np.linalg.matrix_rank(Sigma_hat)
    full_rank = (rank == p)

    cond_numbers.append(cond)
    ranks.append(rank)

    flag = '' if full_rank else ' <- 🧠 奇异!'
    print(f'{p:>8} {p/T:>6.3f} {cond:>14.1f} {rank:>8} {str(full_rank):>8}{flag}')

print('\n📊 结论:')
print(' 当 p 从5增加到200, 条件数从百量级爆炸到亿量级以上')
print(' 当 p >= T, 矩阵秩 < p, 变成奇异矩阵—马科维茨优化彻底崩溃!')

```

```

In [ ]: # — 可视化：条件数随维度的爆炸式增长 —————

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 左图：条件数 (对数坐标)
axes[0].semilogy(dimensions, cond_numbers, 'o-', color='#E74C3C', linewidth=2, markersize=10)
axes[0].set_xlabel('股票数量 p', fontsize=12)
axes[0].set_ylabel('协方差矩阵条件数 (对数)', fontsize=11)
axes[0].set_title('条件数随维度爆炸式增长', fontsize=13, fontweight='bold')
axes[0].axvline(x=T, color='gray', linestyle='--', alpha=0.7, label=f'T={T} (奇异矩阵)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 右图：有效秩比例
rank_ratio = [r / p for r, p in zip(ranks, dimensions)]
bar_colors = ['#27AE60' if r == 1.0 else '#E74C3C' for r in rank_ratio]
axes[1].bar(range(len(dimensions)), rank_ratio, color=bar_colors, alpha=0.8)
axes[1].set_xticks(range(len(dimensions)))
axes[1].set_xticklabels([str(d) for d in dimensions])
axes[1].set_xlabel('股票数量 p', fontsize=12)
axes[1].set_ylabel('有效秩 / p (越低越糟糕)', fontsize=11)
axes[1].set_title('矩阵秩随维度衰减', fontsize=13, fontweight='bold')
axes[1].axhline(y=1.0, color='green', linestyle='--', alpha=0.5, label='满秩 (理想)')
axes[1].legend()
axes[1].grid(True, alpha=0.3, axis='y')

plt.suptitle('高维协方差矩阵的「维度诅咒」——马科维茨的软肋', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('\n谭博士盯着这张图, 盯了很久很久。')
print('「我加了50只股票, 以为是分散化, 实际上是在用一个残缺的协方差矩阵做优化...」')

```

第三章：绝处逢生

2023年9月，上财图书馆，深夜

林总那边的压力越来越大。苏阿姨的代理律师已经发来了一封措辞严峻的邮件。

谭博士请了年假，把自己关在上财图书馆里。

他知道，真正的问题不是市场，而是**协方差矩阵的估计**。样本协方差矩阵在高维情形下，就像一张被撑破了的网——洞越来越多，鱼全漏走了。

他翻遍了所有能找到的高维统计文献，越看越绝望。凌晨三点，图书馆的灯只剩一半还亮着。

手机突然震动了。

不是消息提示——是来电。

张教授。

谭博士愣了一秒，接起来：「教授？您.....还没睡？」

「我这个人，睡得少。」张教授的声音带着一贯的平静，没有丝毫睡意，「你上周在学院群里发了一条消息，说在做高维协方差估计遇到问题。我今天才看到。说说，什么情况？」

谭博士一口气把事情经过说完——股票池从10只扩到50只，协方差矩阵崩溃，组合回撤，苏阿姨的电话。

电话那头沉默了几秒。

「你现在用的是样本协方差矩阵？」

「对。」

「那崩溃是必然的，」张教授说，语气平静，像在讲一道已经解好的题，「 $p = 50$, $T = 252$ ，协方差估计严重欠定。但样本量不够，只是一半的问题。」

谭博士握着手机：「另一半是什么？」

「另一半是——你有高频数据，但你没用。」

谭博士愣了一下。

「你知道 Fan Jianqing 吗？」张教授继续说。

「知道，普林斯顿的。」

「2012年，他和 Yingying Li、Ke Yu 在 JASA 发了一篇文章。」张教授报出文章标题，「*Vast Volatility Matrix Estimation Using High-Frequency Data for Portfolio Selection*。你现在有电脑吗？」

「有。」

「下载来看看。先看第三节。」

谭博士快速在数据库里搜索，几秒钟后，PDF 出现在屏幕上。

「找到了。」

「你知道高频数据里最麻烦的问题是什么吗？」张教授问，「不是数据多，而是数据乱。不同股票，不在同一时刻交易。苹果可能每秒都有成交，一只小盘股可能五分钟才一笔。你要同时估 50 只股票的协方差，时间戳根本对不上。」

谭博士盯着 PDF 第三节，隐约看到了「refresh time」这个词。

「刷新时间，」张教授说，「当所有股票各自都至少完成了一笔交易，这个时刻叫做一个刷新时间点。用这些时间点来同步价格，既解决了非同步交易的问题，又能估计每一对股票之间的协方差。」

「但高频价格里还有噪声——」谭博士接道。

「对，」张教授的语气轻松了一点，「你看出来了。买卖价差、报价误差，这些都会让高频收益率的方差虚高。Fan 他们用了**两尺度已实现协方差 (TSCV)** 来处理：把数据分成粗尺度和细尺度，两个尺度的差，正好抵消了微结构噪声的偏差。」

「一边解决同步问题，一边解决噪声问题，」谭博士慢慢说，「然后样本量从 252 条变成——」

「变成几万条，」张教授接道，「协方差估计的精度，天壤之别。」

「还有一件事，」张教授补了一句，「他们在投资组合优化里加了一个总仓位约束——多头和空头头寸的绝对值之和不超过 c 倍。这个约束来自 Fan 自己更早的工作，它让组合对协方差估计误差的敏感性大幅下降。你现在的系统有这个约束吗？」

谭博士顿了一下：「没有。」

「加上它，」张教授说，「你会发现，就算协方差估计还有一点误差，组合也不会崩。这才是稳健的系统。」

电话挂断了。

谭博士靠在椅背上，手机还握在手里。窗外，陆家嘴的灯还亮着。

他打开论文第三节，开始读。

Fan, Li & Yu (2012) 的核心思想

问题的根源：高频数据的两个拦路虎

日频数据不够用，这谭博士已经明白了。但高频数据带来两个新问题：

问题	表现	影响
非同步交易	各股票成交时间不一致，无法直接对齐计算协方差	估计出错，甚至无法计算

问题	表现	影响
微结构噪声	买卖价差、报价误差叠加在真实价格上	波动率被高估，协方差失真

Fan et al. 用**两步走**的方法同时解决这两个问题。

第一步：刷新时间（Refresh Time）——同步非同步交易

配对刷新（Pairwise-Refresh）：对每一对股票 (i, j) ，找出两者都各自完成了一笔交易的时刻。

$$\tau_k^{(i,j)} = \min\{t > \tau_{k-1}^{(i,j)} : \text{股票 } i \text{ 和 } j \text{ 都有成交}\}$$

优点：每对股票保留的数据量多，元素级估计精度高 缺点：估计出的矩阵可能**不是正定矩阵**，需要做正定投影

全量刷新（All-Refresh）：等所有 p 只股票都完成一笔交易，记为一个刷新时刻。

优点：结果通常是正定矩阵，直接用于优化 缺点：数据量少（需等最慢的股票成交），精度略低

第二步：两尺度已实现协方差（TSCV）——去除微结构噪声

对同步后的价格序列，Fan et al. 使用 **TSCV（Two-Scale Realized Covariance）**：

$$\hat{\Sigma}_{\text{TSCV}} = \underbrace{[X, Y]_1^{(K)}}_{\text{粗尺度估计}} - \frac{\bar{n}_K}{\bar{n}_J} \underbrace{[X, Y]_1^{(J)}}_{\text{细尺度估计}}$$

分量	作用
粗尺度 $[X, Y]^{(K)}$	捕捉真实协方差信号（对噪声取平均，噪声减弱）
细尺度 $[X, Y]^{(J)}$	主要反映微结构噪声（间隔短，噪声主导）
两尺度之差	真实协方差 $\approx$ 粗尺度 - 细尺度，噪声被对消！

第三步：总仓位约束（Gross-Exposure Constraint）——对抗估计误差

即使 TSCV 大幅改善了协方差估计，仍有残余误差。Fan et al. 引入**总仓位约束**：

$$\sum_{i=1}^p |w_i| \leq c$$

其中 $c = 1$ 表示不允许做空， $c > 1$ 允许一定程度的做空。

这个约束的奇妙之处在于：即使协方差估计有误差 ϵ ，组合实际风险和感知风险之差也被控制在 $c^2\epsilon$ 以内。**约束越紧，系统越稳健。**

为什么高频数据这么有用？

维度	日频 (传统方法)	5分钟高频 (Fan et al.)
每天观测数	1	~ 48 (每交易日)
一年总观测数	252	~ 12,000
协方差估计误差速率	$O(p/\sqrt{T})$ 慢	$O((\log p)^\alpha / \tilde{n}^{1/6})$ 快
能捕捉的时变结构	长期平均	本地波动率 (短期动态)

核心洞见：高频数据不只是「更多数据」，更重要的是**缩短了估计窗口**，让协方差矩阵能反映近期的时变结构，而不是被长期平均淹没。

```
In [ ]: # — 直观演示：观测量对协方差估计精度的影响 —————
# 用蒙特卡洛模拟展示：T 越大，估计越准

np.random.seed(2024)
p = 20 # 固定股票数 = 20

# 构造「真实」协方差矩阵 (3因子结构)
B = np.random.randn(p, 3) * 0.05
Lambda = np.diag([0.04, 0.02, 0.01])
Sigma_u = np.diag(np.random.uniform(0.0001, 0.0009, p))
Sigma_true = B @ Lambda @ B.T + Sigma_u

# 不同数据频率对应的样本量
sample_sizes = [
    ('日频 1年', 252),
    ('日频 3年', 756),
    ('5分钟 1年', 252 * 48),
    ('1分钟 1年', 252 * 240),
]

print(f'股票数量 p = {p}, 比较不同数据频率下的协方差估计误差 (100次蒙特卡洛均值) \n')
print(f'{"数据频率":>12} {"观测数 T":>10} {"p/T":>7} {"相对误差 (Frobenius)":>10}')
print('-' * 60)

for label, T_eff in sample_sizes:
    errors = []
    for _ in range(100):
        R = np.random.multivariate_normal(np.zeros(p), Sigma_true, T_eff)
        Sig_hat = np.cov(R.T)
        err = np.linalg.norm(Sig_hat - Sigma_true, 'fro') / np.linalg.norm(Sigma_true, 'fro')
        errors.append(err)
    mean_err = np.mean(errors)
    bar = '█' * int(mean_err * 50)
    print(f'{"label":>12} {"T_eff":>10,} {"p/T_eff":>7.4f} {"mean_err":>8.4f} {"bar}')
```

```
print('\n💡 结论：观测越多（高频数据），协方差矩阵估计越准确！')
```

```
print('Fan et al. (2012) 将高频数据系统性地引入投资组合优化—这是关键贡献。')
```

天色开始泛白。

图书馆的保安敲了敲门：「同学，要关门了。」

谭博士抬起头，才发现外面天空已经从黑色变成了深蓝色，东方隐约透出一丝亮光。他的笔记本上密密麻麻写满了公式和步骤——刷新时间、TSCV、总仓位约束，PDF 的第三节他读了三遍。

他发了一条消息给张教授：

「教授，读完了。刷新时间+TSCV+总仓位约束，三件事联动，才能在高维情形下做稳健的组合优化。思路通了。谢谢您今晚。」

回复很快，一如既往：

「去睡一觉，睡醒了来学院找我。带上你的数据，我们仔细聊聊怎么把这套方法接进你的 Agent 系统里。——张」

谭博士收起手机，走出图书馆。秋天的风带着微微的凉意，路灯正在一盏一盏熄灭。

他感觉心里热的。

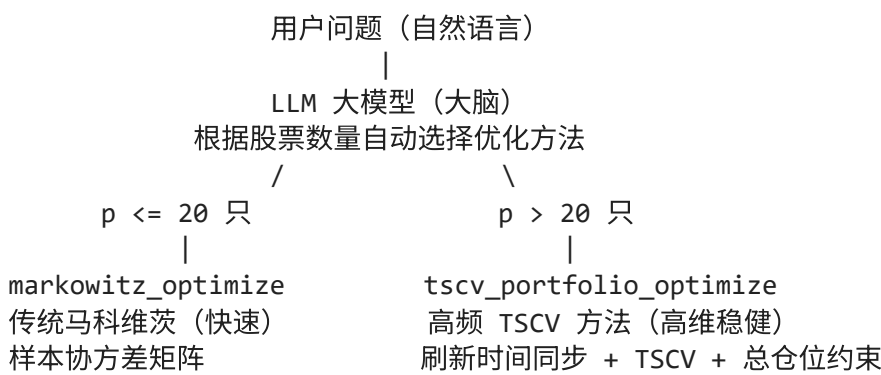
第四章：重建神殿

2023年10月，正思资本

接下来的三周，谭博士几乎没有回家。

他要重新设计 Agent 系统——不只是修一个漏洞，而是在原有架构上，加入一个全新的工具：一个基于 Fan, Li & Yu (2012) 方法的高频协方差矩阵估计工具。

新的系统架构：



关键设计决策：让 LLM 根据问题的特征自动选择工具——这正是 Agent 相比传统程序的核心优势。

🎓 **教学重点：**下面展示的新工具是一个「骨架设计」（Skeleton）。完整的高频数据处理和 TSCV 计算实现，是后续课程的内容。

但请注意：**工具的「接口设计」和「Docstring」才是 Agent 开发的灵魂**——正确描述工具的用途和适用场景，LLM 才能做出正确的调用决策。

```
In [ ]: # — Tool 3 (新) : Fan, Li & Yu (2012) 高频 TSCV 协方差估计工具 —
#
# 注意：这是一个精心设计的「骨架工具」
# 重点在于：
#   1. Docstring 的写法—告诉 LLM 何时选择这个工具（而非 markowitz）
#   2. 接口设计—参数如何设计才能让 LLM 正确传参
#   3. TODO 注释—展示完整实现的思路

@tool
def tscv_portfolio_optimize(
    gross_exposure: float = 1.5,
    sync_method: str = 'pairwise',
    risk_free_rate: float = 0.02,
) -> str:
    '''使用 Fan, Li & Yu (2012) JASA 方法进行高维投资组合优化。

    当股票数量较多（通常  $p > 20$ ）时，传统马科维茨方法的样本协方差矩阵
    精度急剧下降。本工具采用三步框架在高维情形下提升协方差估计质量：

    第一步：刷新时间同步（Refresh-Time Synchronization）
        处理高频数据中的非同步交易问题。两种策略：
        - pairwise（配对刷新）：对每对股票单独同步，保留更多数据，精度更高
        - all（全量刷新）：等所有股票都完成一笔交易，结果直接正定

    第二步：两尺度已实现协方差（Two-Scale Realized Covariance, TSCV）
        同时消除微结构噪声（买卖价差、报价误差）的影响。
        
$$\text{Sigma\_TSCV} = \text{粗尺度估计} - (n_{\text{coarse}}/n_{\text{fine}}) * \text{细尺度估计}$$


    第三步：总仓位约束优化（Gross-Exposure Constrained Portfolio）
        在协方差矩阵 Sigma_TSCV 下，求解带总仓位约束的最小风险组合：
        
$$\min w^T \text{Sigma\_TSCV} w$$

        s.t.  $\sum(w) = 1, \sum(|w_i|) \leq \text{gross\_exposure}$ 

    适用场景：
    - 股票池规模较大 ( $p > 20$  只)
    - 需要捕捉短期时变波动结构（如高波动市场）
    - 对传统马科维茨结果稳健性有要求时

    请先调用 download_stock_data 下载基础数据。

    Args:
        gross_exposure: 总仓位约束上限 c，默认 1.5（允许 25% 净空头）。
            c=1.0 表示不允许做空；c 越大，允许做空越多，
            但组合对估计误差也越敏感。
        sync_method: 同步策略，'pairwise' 或 'all'，默认 'pairwise'。
        risk_free_rate: 年化无风险利率，默认 0.02（2%）。美股建议 0.045。

    Returns:
        基于 TSCV 协方差估计的最优投资组合权重及风险指标（JSON 字符串）
        ...

    # =====
    # TODO：待实现以下步骤（下一阶段课程内容）
    # -----
    # Step 1: 下载高频日内数据（5分钟 K 线）
    #         A 股: akshare.stock_zh_a_hist_min_em(symbol, period='5')
    #         美股: yfinance.download(ticker, interval='5m')
    #
    # Step 2: 刷新时间同步
    #         if sync_method == 'pairwise':
```



```

#           对每对 (i, j) 计算配对刷新时间序列
#           用 TSCV 估计 sigma_hat[i][j] (元素级)
#           拼成矩阵后做正定投影 (Sigma -> Sigma+)
#           else: # 'all'
#               计算全量刷新时间序列 (所有股票均成交的最小时刻集合)
#               对全量刷新后的价格序列直接应用 TSCV
#
# Step 3: TSCV 计算 (以全量刷新为例)
#           n_obs = 同步后观测数 (远大于252)
#           K_opt = round(n_obs ** (2/3)) # 最优粗尺度
#           coarse = realized_covariance(prices, K=K_opt) # 粗尺度
#           fine = realized_covariance(prices, K=1) # 细尺度
#           Sigma_tscv = coarse - (K_opt / n_obs) * fine
#
# Step 4: 总仓位约束下求解最优权重
#           from scipy.optimize import minimize
#           constraints = [
#               {'type': 'eq', 'fun': lambda w: np.sum(w) - 1},
#               {'type': 'ineq', 'fun': lambda w: gross_exposure - np.sum(np.a
#           ]
#           res = minimize(lambda w: w @ Sigma_tscv @ w, w0,
#                           method='SLSQP', bounds=None, constraints=constraint
#           =====

return json.dumps({
    '状态': '此工具正在开发中, 即将上线',
    '方法': 'Fan, Li & Yu (2012) 刷新时间 + TSCV + 总仓位约束',
    '参考论文': 'Vast Volatility Matrix Estimation Using High-Frequency Data',
    '期刊': 'Journal of the American Statistical Association, 107(497), 412-4',
    '同步策略': sync_method,
    '总仓位约束': gross_exposure,
    '说明': '完整实现需要高频日内数据接口, 下一阶段课程将带领大家逐步完成! ',
}, ensure_ascii=False, indent=2)

print('✅ Tool 3 [tscv_portfolio_optimize] 骨架设计完成')
print('Docstring 中明确了适用场景 (p > 20) 和方法原理')
print('LLM 会根据 Docstring 自动在适当场景选择此工具')

```

```

In [ ]: # — 重建 Agent: 三工具新版本 —————
NEW_SYSTEM_PROMPT = '''你是一位专业的量化投资顾问, 擅长根据股票组合的规模和特征,
选择最适合的投资组合优化方法。

你有三个工具可以调用:
1. download_stock_data: 下载股票低频 (日频) 历史行情数据
2. markowitz_optimize: 传统马科维茨均值-方差优化 (适合股票数 p <= 20 的小组合)
3. tscv_portfolio_optimize: Fan, Li & Yu (2012) 高频 TSCV 方法
   (适合股票数 p > 20 的大组合, 或需要捕捉短期时变波动结构时)
   使用刷新时间同步处理非同步交易, TSCV 去除微结构噪声, 总仓位约束提升稳健性

工作流程:
1. 调用 download_stock_data 下载数据
2. 根据股票数量判断优化方法:
   - p <= 20: 调用 markowitz_optimize (传统方法, 简单快速)
   - p > 20: 调用 tscv_portfolio_optimize (高频方法, 高维稳健)
3. 解读结果, 从「风险收益特征」和「适合人群」两个维度给出配置建议

注意: 本分析仅为教学演示, 不构成实际投资建议。'''

```

```
new_agent = create_agent(  
    model=llm,  
    tools=[  
        download_stock_data,  
        markowitz_optimize,  
        tscv_portfolio_optimize, # <- 新增! 高频 TSCV 工具  
    ],  
    system_prompt=NEW_SYSTEM_PROMPT,  
)  
  
print('✅ 新版 Agent 创建成功! ')  
tool_names = [t.name for t in [download_stock_data, markowitz_optimize, tscv_por  
print(f' 已注册工具: {tool_names} ')  
print()  
print('💡 设计亮点: ')  
print('    -> 当股票数 <= 20, LLM 选择 markowitz_optimize (样本协方差, 简单直接) ')  
print('    -> 当股票数 > 20, LLM 选择 tscv_portfolio_optimize (刷新时间+TSCV, 高维稳
```

第五章：涅槃重生

2023年12月，正思资本会议室

苏阿姨坐在对面，表情依然严肃。林总坐在她旁边，捧着咖啡。

谭博士打开了他的笔记本，深吸一口气。

「苏阿姨，林总，今天我来汇报两件事：第一，8月的亏损，我已经找到了根本原因。第二，我们的系统已经完成了升级。」

接下来的二十分钟，他把一切都讲清楚了——维度诅咒、协方差矩阵的崩溃、Fan, Li & Yu 2012年的论文、刷新时间同步与 TSCV 方法、新的三工具 Agent 架构。

苏阿姨没有打断他。等他说完，沉默了一会儿，然后说：

「你知道我为什么选正思吗？不是因为他们从来不犯错。是因为他们犯了错，能搞清楚为什么犯、怎么不再犯。」

她顿了顿，「继续干。」

谭博士感到一股暖流涌上来，有点想笑，又有点想哭。

林总放下咖啡，拍了拍他的肩膀，什么都没说。

春节前夕，上海财经大学，统计学院

会议结束后第二周，谭博士请了半天假，带着一盒新茶和一份打印好的系统报告，专程回到上财。

张教授的办公室在统计学院四楼，门开着。他坐在桌后，面前摆着一叠学生的论文，手里拿着红笔，眼神专注。

「进来吧。」他没抬头，只是说。

谭博士把报告放在他桌上。张教授拿起报告，一页一页翻看——新系统的回测曲线，三工具 Agent 的架构图，以及和旧系统的对比数据。

沉默了很久。

「你用 TSCV 做的协方差估计，收敛了吗？」他指着一张图问。

「收敛了，最大元素估计误差从日频的十倍量级降到了两倍以内，组合风险可控多了。」

张教授点点头，把报告合上：「你做对了一件事——不是把刷新时间和 TSCV 当成黑箱套用，而是真正理解了它们为什么有效。这才是研究的态度。」

谭博士有点不好意思：「当初您讲 Fan 老师论文的背景，我才明白，原来问题的定义比解法更难。」

张教授抬起头，看了他一眼：「这句话，你三年前入学的时候，我就说过。」

谭博士一愣，然后笑了。

「我那时候以为是套话。」

「所有的道理，都是套话，」张教授说，把红笔放下，「直到你自己踩坑了，才变成真话。你这次踩得值。」

谭博士坐在对面，只能感慨：“上财统院的课，真是一节都不能旷啊！”。

课程总结

LangChain Agent 设计的五条心法

#	原则	谭博士的教训
1	Docstring 是工具的灵魂	写清楚「用途、参数、适用场景」，LLM 才能正确调用
2	System Prompt 定义业务逻辑	何时用哪个工具，应在 System Prompt 中明确告知 LLM
3	每个工具做单一事情	<code>download</code> 和 <code>optimize</code> 分开，各司其职，便于组合
4	新需求 = 新增工具	不改原有工具，新增工具并更新 Agent 配置
5	错误信息要对 LLM 友好	工具出错时用自然语言解释原因，而非直接抛异常

Fan, Li & Yu (2012) 的学术贡献

维度	传统马科维茨	Fan, Li & Yu (2012) TSCV 方法
数据	日频，252 个观测/年	5分钟高频，约12,000个观测/年

维度	传统马科维茨	Fan, Li & Yu (2012) TSCV 方法
同步问题	无处理（日频对齐即可）	刷新时间 （配对刷新/全量刷新）
噪声问题	无处理	TSCV 两尺度抵消微结构噪声
组合稳健性	无约束，误差易放大	总仓位约束 ，限制误差积累
适用规模	$p < 20$ （小组合）	p 可达数百（大规模组合）
估计误差收敛速率	$O(p/\sqrt{T})$	$O((\log p)^\alpha / \tilde{n}^{1/6})$

代码架构速查

```
from langchain_core.tools import tool
from langchain.agents import create_agent

# 1. 定义工具（关键：写好 Docstring! ）
@tool
def my_tool(param: str) -> str:
    '''工具用途描述（LLM 的唯一学习来源）

    Args:
        param: 参数含义及示例
    Returns:
        返回值格式说明
    '''
    return json.dumps(result, ensure_ascii=False)

# 2. 创建 Agent
agent = create_agent(
    model=llm,
    tools=[my_tool, ...],
    system_prompt='角色设定 + 工作流程 + 工具选择逻辑 + 注意事项'
)

# 3. 调用 Agent
result = agent.invoke({'messages': [HumanMessage(content='用户问题')]}))
print(result['messages'][-1].content)
```

🎓 **写在最后：**谭博士的故事告诉我们，金融 AI 不只是把公式变成代码。

真正的挑战，是在「模型失效」的时候，有勇气和能力找到真正的问题所在。

Fan, Li & Yu (2012) 的洞见——**用刷新时间对齐非同步交易，用 TSCV 抵消微结构噪声，用总仓位约束抵御估计误差**——正是因为有人在真实的市场数据中摸爬滚打，才发现、被验证、被珍视。

这，就是理论与实践的最美距离。